moOngy.
Minds on the move

React Avançado · Módulo 2

Sessão 4

Memoization e React Compiler

qui 12 nov 2026 · 19h–21h

João Gonçalves · joaogoncalves.net

Recap da Sessão 3

Rendering e composição — tag sessao-03

Hooks

- Snapshot → forma funcional
- Derivado: calcula-se no render
- Hooks no topo, sempre pela mesma ordem

Rendering

- Renderiza: state · **pai** · context
- Render → reconciliation → commit
- Keys estáveis · key nova = reset

Composição

- Moving state down → `SearchForm`
- `children` / slots: o que se recebe já feito não renderiza
- `AppShell`: onde vs o quê

Por resolver: **Select** num card faz renderizar **todos** os `FlightCard`. A composição já não chega → hoje: `memo`, `useCallback` e o React Compiler.

Agenda de hoje

✓	**Exercício da Sessão 3**	o bónus: quem pisca?	≈ 3 min
01	**Memoization à mão**	igualdade referencial · memo · useMemo · useCallback	≈ 25 min
02	**Medir e corrigir no Skyline**	Profiler · um memo que não funciona · props estáveis	≈ 40 min
03	**React Compiler**	instalar · o que faz · o que não faz	≈ 35 min

Primeiro **medir**, depois otimizar — e deixar o Compiler fazer o trabalho repetitivo.

Exercício da Sessão 3 · quem pisca?

Resolução completa no repo: curso/exercicios/sessao-03-resolucao.md (no fim desta sessão)

Escrever no From

- Pisca **só** o `SearchForm` (e os `SearchInput`)
- O `filter` desceu: o `FlightSearch` não renderiza

Select num card

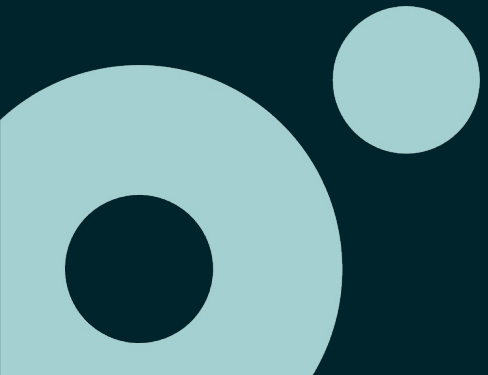
- Piscam o `FlightSearch`, a `FlightList` e **todos** os cards
- O `basket` vive no pai: o pai renderiza → os filhos também

A segunda é o ponto de partida de hoje: como é que só **o card clicado** renderiza?

Bloco 01

Memoization à mão

memo, useMemo e useCallback — o que fazem de facto



Pergunta: estes valores são iguais?

```
const a = { from: 'Lisbon' };
const b = { from: 'Lisbon' };
a === b;          // ?

const f = () => {};
const g = () => {};
f === g;          // ?

'Lisbon' === 'Lisbon';   // ?
```

Pensem antes de responder

- `true` ou `false`?
- E se `a` e `b` fossem criados em **dois renders** do mesmo componente?

Pergunta: estes valores são iguais?

```
const a = { from: 'Lisbon' };
const b = { from: 'Lisbon' };
a === b;          // false

const f = () => {};
const g = () => {};
f === g;          // false

'Lisbon' === 'Lisbon';   // true
```

Resposta

- Objetos e funções comparam-se por **referência**, não por conteúdo
- Em cada render, `{…}` e `() => …` criam valores **novos**
- Primitivos (`5`, `'Lisbon'`, `true`) comparam-se por valor

É por isto que o React acha que as props "mudaram" — mesmo quando o conteúdo é igual.

memo: saltar o render se as props forem iguais

Um componente embrulhado em memo só renderiza quando uma prop muda

```
import { memo } from 'react';

function FlightCardBase({ flight, selected, onToggle }: Props)
{
  // …
}

export const FlightCard = memo(FlightCardBase);
```

O que faz

- Compara cada prop com a anterior (`Object.is`, **shallow**)
- Todas iguais → **salta** o render
- Uma prop nova (objeto, função) → renderiza na mesma

Lembrem-se: com `memo`, as props **passam a contar**. Sem `memo`, um filho renderiza porque o pai renderizou.

`memo` não é de graça: compara props em cada render do pai. Só compensa num filho **pesado**.

useCallback e useMemo: manter a mesma referência

Para dar props estáveis a um componente com memo — ou evitar um cálculo pesado

```tsx
// a MESMA função entre renders
const updatebasket = useCallback(
  (id: number, selected: boolean) =>
    setBasket(b => ({ ...b, [id]: selected })),
  []                        // dependências
);
```

```tsx
// o MESMO valor, até as deps mudarem
const sorted = useMemo(
  () => flights.toSorted(byPrice),
  [flights]                 // dependências
);
```

A reter

- Dependências: quando uma muda, a função/valor é criada **de novo** · `useCallback(fn, d)` = `useMemo(() => fn, d)`
- **Sozinhos não evitam renders** — servem para dar props estáveis a um filho com `memo`

Demo: sem memo vs memo + useCallback

react18-vs-19 → Sessão 2 → memo → Compiler

esquerda · sem memo

```
const select = (id: number) => setSel(id);
<Card key={id} id={id} onSelect={select} />
```

direita · memo + useCallback

```
const select = useCallback(
  (id: number) => setSel(id), []);
<CardMemo key={id} id={id} onSelect={select} />
```

Clicar → **os cards piscam**

Clicar → **só a lista** pisca

Ao vivo: à direita, trocar o `useCallback` por uma arrow normal → os cards **voltam a piscar**

Esquecer um `useCallback` anula o `memo` — **sem erro nenhum**. É o problema que o Compiler resolve.

Quando vale a pena memoizar?

Na dúvida: não — mede primeiro

Sim

- O filho é **pesado** e **mediste** que renderiza sem precisar
- Consegues dar-lhe props **estáveis**
- Listas grandes, gráficos, mapas, editores

Não

- Ainda não mediste
- O filho é barato (a maioria)
- Resolve-se com **composição** (Sessão 3)
- As props mudam sempre de qualquer forma

A ordem é sempre: medir → compor → memoizar.

Bloco 02

Medir e corrigir no Skyline

Profiler, um memo que não funciona — e porquê

React DevTools Profiler: medir antes de otimizar

O Highlight updates mostra quem renderiza; o Profiler mostra quanto tempo e porquê

1. ⚙ do Profiler → ligar **Record why each component rendered**
2. ● **Record**
3. Fazer a interação (ex.: Select num card)
4. ■ **Stop** → flamegraph

Como ler

- **Cinzento** = não renderizou
- **Cor** = renderizou (amarelo = mais lento)
- Clicar num componente → **Why did this render?**

Em dev os tempos são mais altos do que em produção — comparem **antes e depois**, não os números absolutos.



Da teoria ao código

Agora, vamos escrever o código

- Um `FlightCard` **lento de propósito**, para o Profiler ter o que medir
- `memo(FlightCard)`… e ver que **não chega**
- `useCallback` no `updatebasket`… e ver que **ainda não chega**
- O card passa a receber o `onToggle` → só **1** card renderiza

dev/slow.ts · FlightCard.tsx · FlightSearch.tsx · FlightList.tsx

S1 · Um FlightCard lento (só para a demo)

Com 5 cards baratos não há nada para medir — simulamos um card pesado

```ts
// src/dev/slow.ts
export function simulateSlowRender(ms = 20): void {
  const start = performance.now();
  while (performance.now() - start < ms) {
    // ocupa o CPU de propósito
  }
}

// FlightCard.tsx — primeira linha do componente
simulateSlowRender();
```

Medir

- Profiler · **Select** num card
- ≈ **5 × 20 ms**: renderizam todos
- Só para a demo — **sai antes do commit**

Porquê numa função à parte? Fica no próximo bloco: o Compiler não gosta de código impuro no render.

S2 · memo(FlightCard)

O que a teoria diz que resolve…

```tsx
import { memo } from "react";

function FlightCardBase({ flight, selected = false, onSelectedChange }:
FlightCardProps) {
  simulateSlowRender();
  // … igual …
}

export const FlightCard = memo(FlightCardBase);
```

Resultado

- **Nada muda**
- Os 5 cards renderizam
- Why did this render? **Props changed: `onSelectedChange`**

Pergunta: que prop é **nova** em cada render — e quem a cria?

S3 · useCallback no updatebasket

…e o que ainda falta

```tsx
// FlightSearch.tsx
const updatebasket = useCallback((flightId: number, selected: boolean) => {
  setBasket(b => ({ ...b, [flightId]: selected }));
}, []);

// FlightList.tsx — continua assim:
onSelectedChange={ selected => onToggle(flight.id, selected) }
//                ↑ arrow nova, para cada card, em cada render
```

Resultado

- **Ainda nada**
- O `updatebasket` já é estável…
- …mas a arrow no `map` é **nova**

O `memo` só funciona se todas as props forem estáveis — basta uma nova para o anular.

S4 · O card recebe o onToggle

Sem arrow no map: todas as props ficam estáveis

```tsx
// FlightCard.tsx
type FlightCardProps = {
  flight: Flight;
  selected?: boolean;
  onToggle: (flightId: number, selected: boolean) => void;
};
<Button … onClick={() => onToggle(flight.id, !selected)}>

// FlightList.tsx
<FlightCard key={flight.id} flight={flight}
  selected={basket[flight.id] ?? false}
  onToggle={onToggle} />
```

Resultado ✓

- Select → **1** card renderiza
- Profiler: ≈ 20 ms em vez de ≈ 100 ms
- `flight` (mesmo objeto), `selected` (boolean), `onToggle` (estável)

✓ **Paragem segura · commit**

Bloco 03

React Compiler

A memoization feita no build — e o que continua a ser connosco

O que é o React Compiler

Um plugin de build que insere a memoization por nós

Como funciona

- Plugin de **build** (Babel): analisa cada componente e hook
- Memoiza valores, funções e JSX — o trabalho de `useMemo` / `useCallback`
- Exige código que cumpre as **regras do React** (render puro)
- Versão **1.0** estável · não vem ligado no template do Vite

O que muda no dia a dia

- Deixamos de escrever `useCallback` / `useMemo` **por rotina**
- `memo`, `useMemo`, `useCallback` não estão deprecated: ficam como **escape hatch**
- React DevTools: selo **Memo** (com estrela)

Não é magia: o Compiler faz o trabalho **repetitivo** — continuamos a precisar de perceber o que se passa.

Demo: o mesmo código, com e sem Compiler

03-exercicios/compiler-demo · escrever "Go" no campo (2 letras)

Renders	SEM Compiler	COM Compiler
`<App />`	2	**2**
`<List />`	2	**0**
`<Task />`	6	**0**
`Filtering…`	6	**0**

Como correr

- `npm run dev` → COM
- `npm run dev:sem-compiler` → SEM
- Consola aberta: cada log = um render

O código **não tem** `memo`, `useMemo` nem `useCallback` — e com o Compiler porta-se como se tivesse.



Da teoria ao código

Agora, vamos escrever o código

- Instalar o Compiler e ligá-lo no `vite.config.ts`
- Apagar o `useCallback` — o Compiler trata dele
- Ver o selo **Memo** (com estrela) e medir outra vez

package.json · vite.config.ts · FlightSearch.tsx

S5 · Instalar e ligar o Compiler

Três pacotes de desenvolvimento e duas linhas no Vite

```
npm i -D babel-plugin-react-compiler @rolldown/plugin-babel @babel/core
```

vite.config.ts

```ts
import react, { reactCompilerPreset } from '@vitejs/plugin-react'
import babel from '@rolldown/plugin-babel'
import { defineConfig } from 'vite'

export default defineConfig({
  plugins: [react(), babel({ presets: [reactCompilerPreset()] })],
})
```

A reter

- `react()` trata do JSX e do Fast Refresh
- `babel(…)` corre o **Compiler**
- Mudar o config → **reiniciar** o `npm start`

S6 · Apagar o useCallback

O Compiler memoiza o updatebasket sozinho

```
// FlightSearch.tsx — volta a ser uma função normal
function updatebasket(flightId: number, selected: boolean): void {
  setBasket(b => ({ ...b, [flightId]: selected }));
}

// e sai o import do useCallback
import { useState } from "react";
```

Resultado ✓

- Select → continua **1** card
- DevTools: selo **Memo** (com estrela) nos componentes
- O código voltou a ser "React simples"

E o `memo(FlightCard)`? Será que também o podemos apagar? → próximo slide.

O Compiler não salta itens de uma lista

Por isso o memo(FlightCard) fica

Demo	O map depende do que mudou?	O Compiler…	Sem memo
compiler-demo · escrever	não	reutiliza a lista inteira	**0 renders**
Skyline · Select	sim (basket)	refaz a lista inteira	**5 renders**

FlightList.tsx

```
{flights.map((flight) => (
  <FlightCard key={flight.id} flight={flight}
    selected={basket[flight.id] ?? false}   // ← depende do basket
    onToggle={onToggle} />
))}
```

Demo (opcional): tirar o `memo` → Select → **5 cards** → repor

O Compiler guarda a lista inteira. Se ela depende do que mudou, refá-la toda — quem compara card a card é o `memo`.

Código impuro: o Compiler desiste

O render tem de ser puro: mesmas props e state → mesmo resultado

dentro do componente

```
function FlightCard(…) {
  const start = performance.now();   // ✗ impuro
  while (performance.now() - start < 20) {}
  // → o Compiler SALTA este componente
}
```

oxlint avisa: `react(purity)` — «React Compiler skipped optimizing this component»

Impuro no render

- `performance.now()`, `Date.now()`, `Math.random()`
- Mudar variáveis de **fora** do componente
- → resultado diferente com as mesmas props

É por isso que o atraso da demo está numa função à parte, em `src/dev/slow.ts`.

'use no memo': o escape hatch

Uma diretiva — como o 'use strict': uma string que o Compiler lê no build

```
export function FlightSearch() {
  'use no memo';      // ← só este componente
  const [flights, setFlights] = useState<Flight[]>([]);
  // …
}
```

O que faz

- Desliga o Compiler **só neste componente**
- Os filhos continuam compilados — e o `memo` à mão também
- Mas recebem props **novas** → o `memo` deles deixa de servir

Com o nome "use"	O que é	Onde
`'use no memo'`	diretiva, lida pelo Compiler no build	hoje
`use(promise)`	Hook do React 19	Sessão 5
`'use client' / 'use server'`	diretivas dos Server Components	Sessão 12

✓ **Paragem segura · retirar o card lento (`dev/slow.ts`) · commit**

Resumo: memoization e React Compiler

Memoization à mão

- Objetos e funções: igualdade por **referência**
- `memo`: salta se as props forem iguais
- `useCallback` / `useMemo`: dar props **estáveis**

No Skyline

- **Medir** primeiro: Profiler
- Uma prop nova anula o `memo`
- O card recebe o `onToggle` → 1 render

React Compiler

- Faz o `useCallback` / `useMemo` por nós
- Itens de lista: o `memo` continua útil
- Código puro, senão salta · `'use no memo'`

Para a Sessão 5: hooks avançados e custom hooks — `useRef`, closures, `useEffectEvent`, `useFlights`.

Teste-se · 1/5

Pergunta

Um filho sem `memo` recebe uma função com `useCallback`. Renderiza quando o pai renderiza?

```
const onToggle = useCallback(…, []);
<Child onToggle={onToggle} />   // Child SEM memo
```

Pensem antes de responder

- A referência da função muda?
- E isso importa a quem não tem `memo`?

Teste-se · 1/5

Resposta

Um filho sem `memo` recebe uma função com `useCallback`. Renderiza quando o pai renderiza?

Sim — mas não por causa da função: com `useCallback` a referência **é a mesma**.

Filho	Recebe	Renderiza quando o pai renderiza?
sem `memo`	função nova	sim
sem `memo`	`useCallback`	sim — o `useCallback` foi inútil
com `memo`	função nova	sim
com `memo`	`useCallback`	**não** ✓

As props só contam quando há `memo`.

Teste-se · 2/5

Pergunta

Um filho com `memo` recebe `style={{ color: 'red' }}`. Renderiza quando o pai renderiza?

```
const Child = memo(ChildBase);
<Child style={{ color: 'red' }} />
```

Pensem antes de responder

- Quantas vezes é criado aquele objeto?

Teste-se · 2/5

Resposta

Um filho com `memo` recebe `style={{ color: 'red' }}`. Renderiza quando o pai renderiza?

Sim — `{{ color: 'red' }}` é um **objeto novo** em cada render.

```
const redStyle = { color: 'red' };   // 1. fora do componente
<Child color="red" />                 // 2. prop primitiva
useMemo(() => ({ … }), [isError])    // 3. se depender de state
```

Constante → fora do componente · depende de state → `useMemo` · se der, prop primitiva.

Teste-se · 3/5

Pergunta

`useMemo(() => a + b, [a, b])` vale a pena se `a` e `b` forem números pequenos?

```
const soma = useMemo(() => a + b, [a, b]);
```

Pensem antes de responder

- Quanto custa somar dois números?
- E o resultado é um objeto ou um primitivo?

Teste-se · 3/5

Resposta

`useMemo(() => a + b, [a, b])` vale a pena se `a` e `b` forem números pequenos?

Não. Somar custa menos do que o próprio `useMemo` — e `7 === 7`: não há referência a proteger.

`useMemo` só para cálculos pesados ou objetos/arrays que vão para um filho com `memo`.

Teste-se · 4/5

Pergunta

Porque é que o `setBasket` não precisa de ir para as dependências do `useCallback`?

```
const updatebasket = useCallback((id: number, sel: boolean)
=> {
  setBasket(b => ({ ...b, [id]: sel }));
}, []);   // ← vazio, e está certo
```

Pensem antes de responder

- O `setBasket` muda alguma vez?
- E a função lê o `basket` de fora?

Teste-se · 4/5

Resposta

Porque é que o `setBasket` não precisa de ir para as dependências do `useCallback`?

1. O React garante que o `setBasket` é **sempre a mesma função**. 2. A **forma funcional** `b => …` recebe o valor atual — não lê o `basket` de fora.

```
setBasket({ ...basket, [id]: sel })   // ✗ basket nas deps → função nova
setBasket(b => ({ ...b, [id]: sel }))  // ✓ deps vazias → sempre a mesma
```

Forma funcional = dependências vazias = função estável.

Teste-se · 5/5

Pergunta

Com o React Compiler ligado, ainda escreveriam `useCallback` no `updatebasket`?

```
// vite.config.ts
plugins: [react(), babel({ presets:
[reactCompilerPreset()] })]
```

Pensem antes de responder

- O que é que o Compiler faz por nós?
- E o que é que vimos que ele não faz?

Teste-se · 5/5

Resposta

Com o React Compiler ligado, ainda escreveriam `useCallback` no `updateBasket`?

Não. O Compiler trata do `useCallback` e do `useMemo`.

O que continua connosco

- O `memo` onde o pai tem de **recriar** o elemento (itens de um `map` que depende do que mudou) — e se mediram que pesa
- Código **puro** — senão o Compiler salta o componente

Com o Compiler: deixamos de escrever `useCallback` / `useMemo` — o `memo` fica só onde é preciso.

Exercício de revisão

Para casa · no Skyline, a partir da tag sessao-04

Obrigatório · ≈ 20 min

- Profiler: gravar **From**, **Select** e **Search**
- Quem renderiza em cada um?
- Explicar numa frase cada resultado

Extra

- `'use no memo'` no `FlightSearch`
- Select: o que muda?
- Porquê?

Bónus

- Passar `options={{ showPrice: true }}` a cada card
- Ver o `memo` partir — **mesmo com o Compiler**
- Corrigir de duas formas

Enunciado no repo: `curso/exercicios/sessao-04.md` · perguntas e respostas: `curso/resumos/sessao-04.pdf` · antes de entregar: `npx oxlint` e `npm run build` sem erros

moOngy.
Minds on the move

Obrigado · até terça, 17 nov

edu@joaogoncalves.net · WhatsApp +351 910 346 437

`github.com/JoaoGoncalves/skyline · tag sessao-04`